

ASSIGNMENT

Productionizing a Multimodal RAG Application

From Prototype to Production-Grade SaaS / Enterprise Architecture

Domain	Generative AI / Multimodal RAG
Current System	Streamlit + PDF parsing + embeddings + Qdrant + retrieval + multimodal generation
Primary Goal	Design the evolution of the current V1 into a secure, scalable, reliable production product
Recommended Output	Architecture proposal, data model, workflows, retrieval plan, security plan, evaluation plan, and phased roadmap

Full-Stack GenAI Bootcamp - Production Architecture Assignment

1. Assignment Scenario

You currently have a working V1 multimodal RAG application. The Streamlit UI directly calls parsing, ingestion, retrieval, and generation logic. The application can parse PDFs, extract text/OCR/tables/images, create embeddings, index them in Qdrant, retrieve relevant evidence, and generate grounded answers using a multimodal LLM.

Your task is to redesign this prototype as a production-ready product. The design should support multiple users, reliable background processing, scalable storage and retrieval, stronger security, better multimodal retrieval quality, evaluation, observability, and future SaaS/enterprise capabilities.

2. Learning Objectives

- Separate UI concerns from backend services and define clear system boundaries.
- Design asynchronous document ingestion for large and long-running workloads.
- Use appropriate storage systems for original files, metadata, and vectors.
- Design secure multi-user and multi-tenant retrieval with authorization filters.
- Improve retrieval quality using hybrid search and reranking.
- Improve multimodal retrieval for images and tables.
- Design document versioning and incremental re-ingestion.
- Define evaluation, observability, latency, cost, and reliability metrics.
- Create a phased roadmap from the current prototype to a production product.

3. Required Design Tasks

3.1 Separate the Backend

Replace direct Streamlit-to-pipeline calls with a service-oriented architecture. The target flow should be: Frontend -> API Layer -> Ingestion Service / Retrieval Service / Generation Service. Streamlit may remain as an internal, admin, or demo interface, while a public product can use a dedicated frontend such as React/Next.js with a backend API.

3.2 Build Asynchronous Ingestion

Do not block a user request while a large PDF is parsed and indexed. Design the flow as: Upload -> Object Storage -> Job Queue -> Parser Workers -> Embeddings -> Qdrant -> status=ready. The UI should expose meaningful processing states such as uploaded, parsing, chunking, indexing, and ready.

3.3 Add Object Storage and a Metadata Database

Move original PDFs and extracted images away from local disk dependency. Use object storage for binary files and a relational metadata database such as PostgreSQL for users, documents, jobs, versions, and related application metadata. Use Qdrant primarily for vector retrieval and searchable payloads.

3.4 Add Multi-User / Multi-Tenant Support

Include tenant_id, user_id, workspace_id, document_id, and permission-related metadata in the document/vector payload. Design a strategy that avoids creating a separate collection for every user when payload-based partitioning is more appropriate.

3.5 Enforce Access Control During Retrieval

Authorization must be part of the retrieval query, not only hidden in the UI. Every search should be constrained by the authenticated user, tenant, role, workspace, or permission scope so that one user can never retrieve another user's content.

3.6 Upgrade Dense Retrieval to Hybrid Retrieval

The current system uses dense semantic search. Design the next version using Dense + Sparse/BM25 -> RRF -> Reranker -> Top-K. Explain where each stage runs and what problem it solves.

3.7 Make Images Truly Searchable

The current image retrieval is largely dependent on OCR/text. Add image understanding during ingestion, for example: Image -> Vision Summary/Caption -> Embedding -> Qdrant. Also describe a future advanced path using native visual or multivector retrieval such as late-interaction/ColPali-style retrieval.

3.8 Add Table Intelligence

Do not rely only on Markdown representations. Store raw table content, a searchable table summary, headers/schema information, and a normalized structured representation so that exact factual questions about rows, columns, dates, amounts, or entities can be answered reliably.

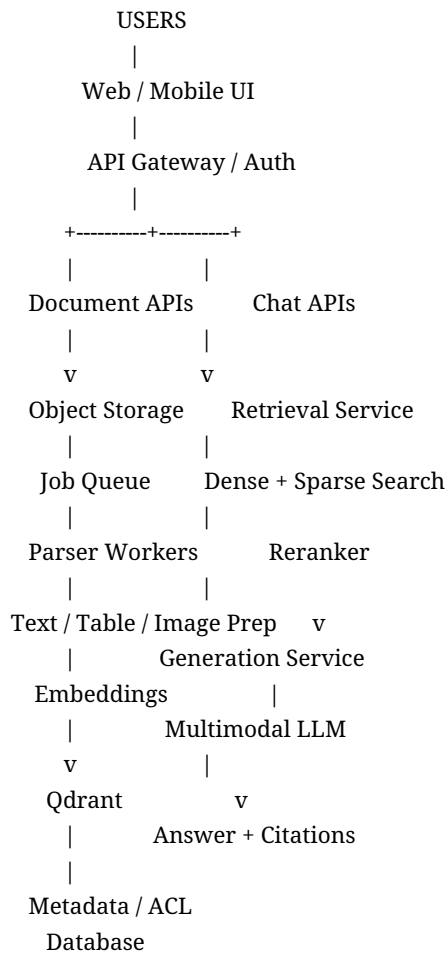
3.9 Add Document Versioning and Incremental Ingestion

Use a document hash/version strategy. If a document is unchanged, skip unnecessary parsing and embedding. If it changes, update or replace only the relevant document data rather than rebuilding the entire workspace.

3.10 Add an Evaluation Framework

Create a repeatable evaluation dataset containing Question, Expected Source, and Expected Answer. Measure retrieval recall/precision, groundedness, answer correctness, citation correctness, latency, token/model cost, and failure rate. The design should make model, prompt, chunking, and retrieval changes comparable across releases.

4. Target Production Architecture



Architecture expectation: Your solution should clearly explain the responsibility of each component, the request/data flow between components, and which parts must scale independently.

5. Product Capabilities to Plan For

- Workspaces and document collections
- Multiple PDFs and multi-document retrieval
- Conversation history
- Source and page preview
- Shareable chats
- User feedback (thumbs up/down)
- Admin analytics
- Connectors such as Google Drive, SharePoint, or S3
- Public/API access
- Audit logs
- Usage quotas
- Billing and subscription controls

6. Recommended Implementation Roadmap

Phase 1 - Current V1	Single-app prototype with parsing, ingestion, dense retrieval, generation, Qdrant, and Streamlit.
Phase 2 - Multi-Document + Authentication	Add user identity, multiple documents, workspace/document ownership, and authenticated access.
Phase 3 - Async Ingestion + Object Storage + PostgreSQL	Move heavy document processing to workers and persist files/metadata in production storage.
Phase 4 - Tenant/RBAC Filtering	Enforce tenant, user, workspace, and role/permission filters during retrieval.
Phase 5 - Hybrid Retrieval + Reranking	Combine dense and sparse retrieval, fuse candidates, rerank, and pass only the best evidence to generation.
Phase 6 - Better Multimodal Retrieval	Add vision summaries/captions, richer table representations, and later native visual/multivector retrieval.
Phase 7 - Evaluation + Observability	Add offline evals, tracing, latency/cost/failure monitoring, feedback loops, and production quality dashboards.
Phase 8 - Production API + Frontend	Expose clean APIs, production authentication, scalable deployment, and a consumer/enterprise-grade frontend.
Phase 9 - Enterprise Integrations + Billing	Add connectors, quotas, auditability, subscription/billing controls, and enterprise governance.

7. Expected Submission

Prepare a concise production-design submission containing the following:

- A high-level production architecture diagram and explanation.
- A component responsibility table covering frontend, API, ingestion, workers, storage, Qdrant, retrieval, reranking, generation, and observability.
- A multi-user / multi-tenant data model showing users, tenants/workspaces, documents, versions, jobs, and permissions.
- An asynchronous ingestion workflow including status transitions and failure/retry handling.
- A retrieval design covering dense, sparse/BM25, fusion/RRF, reranking, metadata filtering, and Top-K generation context.
- An image and table retrieval strategy, including the current limitation and the proposed improved approach.
- A security model for authentication, authorization, tenant isolation, secret management, and auditability.
- A document versioning and incremental-ingestion strategy.
- An evaluation and observability plan with measurable quality, latency, reliability, and cost metrics.
- A phased implementation roadmap identifying what should be built first and why.

8. Evaluation Rubric

Area	Marks	What Good Looks Like	Score
Architecture & Service Separation	20	Clear boundaries, scalable service design, and correct data/request flow.	
Ingestion & Storage Design	15	Async processing, object storage, metadata DB, retries, and versioning.	
Security & Multi-Tenancy	15	Authentication, authorization, tenant isolation, RBAC/ACL-aware retrieval.	
Retrieval Quality	15	Hybrid retrieval, metadata filtering, RRF/fusion, reranking, and evidence selection.	
Multimodal Design	10	Image and table strategies beyond OCR/Markdown-only retrieval.	
Evaluation & Observability	15	Quality metrics, eval datasets, tracing, latency, cost, and failure monitoring.	
Roadmap & Product Thinking	10	Practical sequencing, trade-offs, and readiness for product/enterprise features.	

Key principle: move from a feature demo to an architecture that is secure, scalable, observable, testable, and maintainable.